

Lab 06: Your First Triangle

CP479 Computer Graphics

September 11, 2026

For five labs you have been the GPU: the triangle filler in Lab 03, the matrices in Lab 04, the camera in Lab 05, every pixel of it on one CPU core.

From here on the hardware does that work. You do not write the loop over pixels or the shading; both are given. You write **the data**: five buffers, laid out to the byte, that carry your triangle into the machine. Then you measure what it cost. This is also the first of many Vulkan labs, so the toolchain and the skeleton you learn this week are the ones you keep.

AI coding assistants are permitted and expected. You must be able to explain everything you ship. You must also keep an AI prompt log: for every prompt, record the idea behind it and how you verified the answer (see Section 11). A first triangle is the most copy-pasteable artefact in computer graphics, so be clear about where the work actually is: five buffers, and six write-ups about numbers that do not exist anywhere until *you* have run the measurement on *your* machine. An assistant can hand you the first; nobody can hand you the second.

1 Setup

We install this together in the lecture before the lab, and it is a one-time cost: every lab after this one uses the same toolchain, so nothing here has to be done twice. This section is the list, so you can check what you have or set up a second machine later.

1.1 What you need installed

Five things. Versions are minimums; anything newer is fine.

What	Version	Where
Vulkan SDK	1.3+	https://vulkan.lunarg.com/sdk/home Brings the loader, the validation layers, the shader compilers (<code>glslc</code> , <code>slangc</code>) and GLM . On macOS it is also what gives you Vulkan at all: it installs MoltenVK, which translates Vulkan to Metal. Keep <i>System Global Installation</i> on, and let it set <code>VULKAN_SDK</code> : that is how CMake finds GLM.
CMake	3.24+	https://cmake.org , or <code>brew install cmake</code> , or <code>apt</code> , or bundled with Visual Studio 2022.
GLFW	3.3+	<code>brew install glfw</code> , <code>apt install libglfw3-dev</code> , or <code>vcpkg</code> . The one library that does <i>not</i> come with the SDK, it opens the window.
A Vulkan-capable GPU driver	any	Windows and macOS: nothing extra. Linux: <code>mesa-vulkan-drivers</code> for Intel or AMD, or the proprietary NVIDIA driver.
<i>Linux only</i> , if you used <code>apt</code> rather than the SDK tarball	any	<code>glslang-tools</code> for the shader compiler and <code>libglm-dev</code> for GLM. The SDK tarball already contains both.

What you do *not* need. GLM comes with the SDK, and the only other library is **stb**, two header files vendored in `external/` and used for nothing but comparing what you drew against the reference. Nothing is downloaded when you build. Optional but recommended on Windows: VS Code with the **CMake Tools** extension, which gives you a target list and a run button.

1.2 Check that it worked

```
vulkaninfo --summary
```

You should see your GPU's name and a driver name. On macOS the driver must say **MoltenVK**. If `vulkaninfo` is not found, the SDK is not on your `PATH`: open a new terminal, or on macOS run the SDK's `setup-env.sh`.

1.3 Get the skeleton and build it

<https://github.com/2110479-Computer-Graphics/Lab06-Triangle>

Press *Use this template* to make your own copy, then clone that copy. Your work lives in your repository, so commit as you go. Then run *one* of these. They do the same thing.

```
make info # macOS, Linux
make.bat info # Windows
cmake -S . -B build && cmake --build build --target info # anywhere
# or: VS Code, CMake Tools, pick "info" from the target list and press run
```

`make` is a three-line wrapper around CMake: it configures, builds one target, and runs it. Every part works the same way, `make triangle` and so on, and each part is a separate program, so one that fails to build cannot stop the others.

`./info` is also your troubleshooter. It prints what the skeleton found and, when something is missing, what to do about it:

```
$ make info
device      Apple M2 Pro          api 1.3.334
subgroup    32 lanes
timestamps  1 ns per tick
statistics  pipeline statistics unsupported
student.cpp 0 vertices, stride 0, 0 attributes
TASK 1 incomplete: TRIANGLE is empty
```

Two lines matter later: *timestamps* must be a number, and *subgroup* is a figure you need in Part III. Keep this screenshot for your report.

2 The skeleton

2.1 Files

- **student.cpp: the only file you edit.** Five TASKs.
- **src/utils/Student.h:** declares everything **student.cpp** must define. Read it first; it is the contract for all five TASKs.
- **src/triangle.cpp, src/cost.cpp, src/stripes.cpp, src/earlyz.cpp, src/index.cpp, src/probe.cpp:** one program per part, about 80 lines each, named after what it measures rather than which part uses it. Complete; read them, each is one experiment on one screen.
- **src/core/** and **src/utils/:** device, buffers, render target, pipeline, GPU timer, window, image comparison. Complete; do not edit.
- **shaders/:** twelve GLSL files. Complete: read them, you need not edit them, though if you do the build recompiles them. CMake compiles every **.vert**, **.frag** and **.comp** here, so a shader you add later is picked up automatically. You will need that from the next lab on.
- **external/:** where the one third-party library lands. **stb**, two headers, used only to compare what you drew against the reference. CMake downloads them on the first configure and checks their hash; if your network blocks that, run **external/fetch.sh** on macOS or Linux, or **external/fetch.bat** on Windows. Each is two **curl** commands. GLM comes from the Vulkan SDK.
- **reference/**, **report/:** reference images, and your report with its six write-ups outlined. **predictions.md** in the root is Part II's, filled in before you measure.

3 The pipeline, and the nine objects

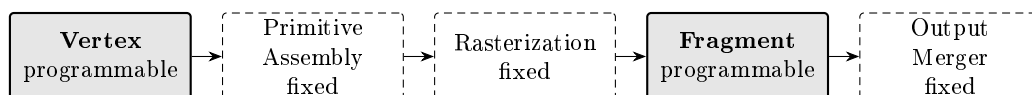


Figure 1: The five stages of the graphics pipeline. Two are programmable; three are fixed-function, and you can only configure them.

3.1 The nine objects the skeleton creates

In **src/core/**, in this order. You write none of them, but two read data you wrote.

Object	What it is for
1 <code>VkInstance</code>	The connection to the loader; where layers are enabled.
2 <code>VkPhysicalDevice</code>	The GPU, and the source of every number <code>./info</code> prints.
3 <code>VkDevice</code> + queue	Your handle for issuing work.
4 <code>Render target</code> + <code>VkSwapchainKHR</code>	You draw into a fixed 800×600 off-screen image, which is copied to the window. Every number in this lab therefore comes from the same resolution.
5 <code>VkRenderPass</code>	Which attachments exist and what happens to them.
6 <code>VkShaderModule</code> $\times 2$	The two shaders, compiled to SPIR-V by the build.
7 Descriptor set layout, pool, set	What makes your TASK 2 buffer visible to a shader.
8 <code>VkPipeline</code>	All of the above plus your TASK 1 vertex input description , frozen into one object.
9 <code>VkCommandBuffer</code>	The recorded draw commands, submitted once per frame.

3.2 Reading the chapter's listings

Every code sample in §6.2 of your chapter is WebGPU. The field names differ from Vulkan's; the numbers do not.

Chapter 6, §6.2.1 (WebGPU)	This lab (Vulkan)
<code>arrayStride: 32</code>	<code>VkVertexInputBindingDescription::stride</code>
<code>{shaderLocation, offset, format}</code>	<code>VkVertexInputAttributeDescription</code>
<code>createQuerySet({type: 'timestamp'})</code>	<code>vkCmdWriteTimestamp</code> into a <code>VkQueryPool</code>

4 The shaders you are given

Listing 1: `shaders/triangle.vert`

```

1 #version 450
2
3 layout(location = 0) in vec2 inPosition;
4 layout(location = 1) in vec3 inColor;
5
6 layout(location = 0) out vec3 fragColor;
7
8 layout(set = 0, binding = 0) uniform Params {
9     mat4 mvp;
10    uint costLoops;
11    uint stripeWidth;
12 } u;
13
14 void main() {
15     gl_Position = u.mvp * vec4(inPosition, 0.0, 1.0);
16     fragColor = inColor;
17 }
```

Listing 2: shaders/triangle.frag

```

1 #version 450
2
3 layout(location = 0) in vec3 fragColor;
4
5 layout(location = 0) out vec4 outColor;
6
7 void main() {
8     outColor = vec4(fragColor, 1.0);
9 }

```

The vertex shader is your Lab 04–05 matrices applied per vertex; the fragment shader is the body of your Lab 03 per-pixel loop. Neither blends anything, yet your triangle will have a gradient across it. Write-up 1 asks where that came from. Note also that there is no y flip: Vulkan’s clip space already points y down, so the flip `viewportTransform()` did for you in Lab 05 is now the hardware’s convention.

$$f(P) = \lambda_0 f(V_0) + \lambda_1 f(V_1) + \lambda_2 f(V_2), \quad \lambda_0 + \lambda_1 + \lambda_2 = 1 \quad (1)$$

Why there is a compile step. In OpenGL you handed the driver GLSL text. Vulkan will not: `vkCreateShaderModule` takes SPIR-V, a binary format, and nothing else. The compiler that used to live in every vendor’s driver is now `glslc`, running at build time. That is one concrete case of §6.4’s *implicit* versus *explicit*, and it is why a shader that fails to compile fails on your desk rather than on a user’s.

5 Part I: Make it appear

Run `make triangle`. It prints a panel, then opens a black window: no data has reached the GPU yet. Two TASKs fix that. Close the window to exit.

5.1 The vertex, to the byte

```

1 struct Vertex {
2     glm::vec2 pos;           // offset 0 .. 7
3     glm::vec3 color;        // offset 8 .. 19
4 };
5
6 static_assert(sizeof(Vertex) == 20, "Vertex must be 20 bytes");
7 static_assert(offsetof(Vertex, pos) == 0, "pos must start at byte 0");
8 static_assert(offsetof(Vertex, color) == 8, "color must start at byte 8");

```

That is `src/utils/Vertex.h`, given: two floats and three floats, no padding, 20 bytes. The two functions below are declared there and defined by you.

Attribute	Bytes	Offset	Vulkan format
pos	8	0	VK_FORMAT_R32G32_SFLOAT
color	12	8	VK_FORMAT_R32G32B32_SFLOAT
stride	20		

A `glm::vec3` is 12 bytes *here*, in a vertex buffer, because vertex fetch reads exactly what your `format` and `offset` say. Inside a uniform block the same type obeys `std140` and rounds up to 16. Same type, two layout rules, decided by where the bytes go. TASK 2 is on the other side of that line.

5.2 TASK 1: TRIANGLE and the two description functions

```
1  const std::vector<Vertex> TRIANGLE = {
2      // { { x, y }, { r, g, b } },
3  };
4
5  VkVertexInputBindingDescription Vertex::bindingDescription() {
6      VkVertexInputBindingDescription desc{};
7      desc.binding = 0;
8      desc.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
9      desc.stride = 0; // TODO(TASK 1b): bytes from one vertex to the
      next
10     return desc;
11 }
12
13 std::vector<VkVertexInputAttributeDescription> Vertex::
14     attributeDescriptions() {
15     // TODO(TASK 1c): two attributes. Each needs binding, location, format
16     // , offset.
17     return {};
18 }
```

Check values.

- $V_0 = (0.0, -0.5)$ red, the **apex at the top of the screen**. In Vulkan clip space $y = -1$ is the top, so the apex has a *negative y*. Write +0.5 out of OpenGL habit and your triangle points down *and* disappears.
- $V_1 = (-0.5, 0.5)$ green, bottom **left**. $V_2 = (0.5, 0.5)$ blue, bottom **right**.
- **That order matters**. Part I culls back faces with front-facing set to COUNTER_CLOCKWISE, and facing is decided in framebuffer coordinates, where y points down: the order that looks clockwise on paper is the front-facing one. Swap V_1 and V_2 and the triangle vanishes. Try it once, deliberately.
- `stride = 20`, offsets 0 and 8. The panel echoes what it received: `stride 20, offsets [0 8 ]`.

TASK 1 asks for the offsets as literals so you work them out once. In production you would write `offsetof(Vertex, color)`.

5.3 TASK 2: uniformBlock()

The vertex shader reads its matrix from a uniform buffer. `std140` gives a `mat4` 64 bytes and rounds the block up to a multiple of 16.

Member	GLSL type	Offset	Size
<code>mvp</code>	<code>mat4</code>	0	64
<code>costLoops</code>	<code>uint</code>	64	4
<code>stripeWidth</code>	<code>uint</code>	68	4
block size, padded			80

```
1  struct Params {
2      std::uint32_t placeholder[20]; // TODO(TASK 2a): three std140 members
3      // , padded
4  };
```

```

4
5 static_assert(sizeof(Params) == 80, "Params must match the std140 table"
6 );
7 std::vector<std::uint8_t> uniformBlock(std::uint32_t costLoops,
8                                         std::uint32_t stripeWidth) {
9     Params params{};
10    // TODO(TASK 2b): identity matrix, glm::mat4(1.0f), and the two knobs.
11
12    std::vector<std::uint8_t> bytes(sizeof(Params));
13    std::memcpy(bytes.data(), &params, sizeof(Params));
14    return bytes;
15 }

```

`mvp` is the identity for Part I. Parts II and III drive `costLoops` and `stripeWidth` through this same function, so the buffer you write here is the instrument for both.

5.4 Expected result

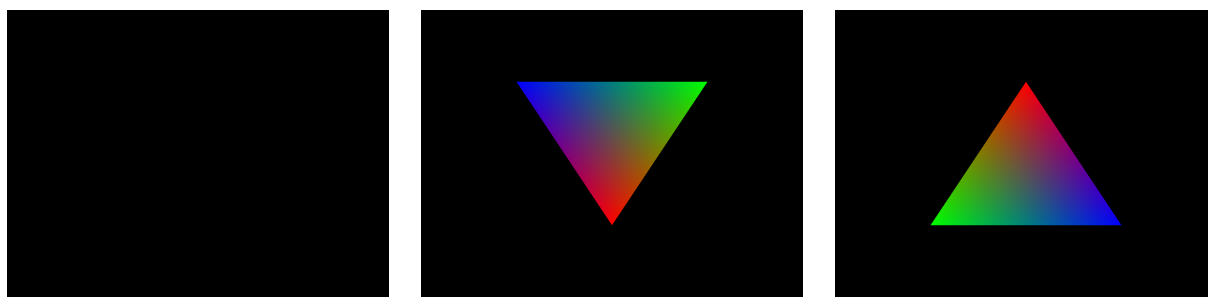


Figure 2: Three states of the same program. **Left:** the skeleton, nothing drawn, panel naming TASK 1a, 1b and 1c. **Middle:** TASK 1 done with the apex at $y = +0.5$, the OpenGL habit: Vulkan puts $y = -1$ at the top, so it points down. **Right:** expected. Red apex, green bottom left, blue bottom right, gray where all three meet.

vertex buffer	3 vertices, stride 20, offsets [0 8]
uniform block	80 bytes
rasterized	12.5% of the target
centroid	pixel (400, 350) reads (85, 85, 86)
image	MATCH
gpu time	0.009 ms median, 0.134 ms p95

Two checks needing no tools. `rasterized` must read 12.5%: corners at ± 0.5 cover one eighth of the clip-space square, which you can work out on paper. And a color picker on the middle of the triangle must read (85,85,85), give or take one: Equation 1 with $\lambda_0 = \lambda_1 = \lambda_2 = \frac{1}{3}$, so $\frac{1}{3}$ of 255. The target is a UNORM format, so no sRGB curve sits between the shader and your picker.

5.5 Diagnostics

Each symptom has one cause, and all of them are *data* bugs: the shaders are already correct.

What you see	What it is
An incomplete list naming TASK 1a, 1b or 1c	The skeleton before you start. It reports rather than crashing.
rasterized 0.0% and a <i>winding</i> hint	Back-face culled. Swap any two vertices. Write the apex at $y = +0.5$ as well and you have both faults at once, which is why the reflexive OpenGL triangle shows nothing here.
rasterized 0.0% and a <i>uniform block</i> hint	TASK 1 done, TASK 2 not: the matrix is all zeros, so every vertex collapses onto the origin.
The triangle points <i>down</i>	Right arithmetic, wrong convention: y is down.
A smaller or skewed triangle, rasterized under 12.5%	stride is wrong, so each vertex takes its position from its neighbor's bytes. Stride 12 instead of 20 draws a wedge at about 3.1%.
Right shape, wrong colors, still 12.5%	The color attribute's offset is wrong while the position's is right. With it set to 0 the center reads (0,43,85).

Write-up 1.

1. Neither shader blends anything, yet the triangle has a gradient. Which stage produced it? Name the stage, the equation, and the Lab 03 function of yours that did the same job.
2. Your Lab 05 `viewportTransform()` flipped y ; the given vertex shader does not. Where did the flip go?
3. Report your center-pixel value and the value Equation 1 predicts.

6 Part II: What did it cost?

Run `make cost`. Nothing to write: the instrument is the uniform buffer you built in TASK 2. Two given shader pairs put the *same* loop in different stages.

Listing 3: `shaders/cost.frag`, and `cost.vert` carries the identical body in the vertex stage

```

1 void main() {
2     float x = fragColor.r;
3     if (scene.fragmentLoop != 0u) {
4         for (uint i = 0u; i < u.costLoops; i++) { x = x * 1.000001 +
5             1.0; }
6     }
7     outColor = vec4(fragColor * clamp(x * 1e-9, 0.0, 1.0) + fragColor,
8         1.0);
9 }
```

The loop feeds the output on purpose: otherwise the compiler deletes it and you measure nothing. A push constant picks the stage, so one pipeline serves both placements. `make cost` runs three scenes in both, six cells.

Scene	Vertex inv.	Isolates
1 triangle, $0.04\times$ scale	3	neither stage
1 triangle, fullscreen	3	the fragment stage
100k instances, $0.04\times$	300,000	the vertex stage

Fragment invocations are not listed: work them out from the coverage the program prints, and say how you did it.

Write-up 2. Predict the six cells in `predictions.md` first, then run at three loop counts.

1. Which scene is expensive in which placement?
2. The loop body is identical in both files. Explain the ratio between your two largest numbers using only the invocation counts above.
3. Your smallest cell is near zero. What is the smallest difference your measurement can resolve, and how do you know?

7 Part III: How wide is a warp, in pixels?

Run `make stripes`. Again nothing to write: `stripeWidth` is the second knob in your TASK 2 buffer. Both branches below run the same instructions, so the only thing that changes is *which pixels* take which one.

Listing 4: `shaders/stripes.frag`

```
1 void main() {
2     uint stripe = uint(gl_FragCoord.x) / max(u.stripeWidth, 1u);
3     float x = fragColor.r;
4     if ((stripe & 1u) == 0u) {
5         for (uint i = 0u; i < u.costLoops; i++) { x = x * 1.000001 +
6             1.0; }
7     } else {
8         for (uint i = 0u; i < u.costLoops; i++) { x = x * 1.000002 +
9             1.0; }
10    }
11    outColor = vec4(fragColor * clamp(x * 1e-9, 0.0, 1.0) + fragColor,
12                    1.0);
13 }
```

`make stripes` sweeps stripe widths of 1, 2, 4, 8, 16, 32, 64, 128 pixels, plus one run with the branch removed. At one pixel per stripe every group of neighboring pixels contains both branches and pays for both; as stripes widen, more groups fall inside one stripe and pay once. Somewhere the curve flattens.

$$\text{Effective throughput} = \frac{\text{active lanes}}{\text{subgroup size}} \times \text{peak throughput} \quad (2)$$

Write-up 3.

1. Plot stripe width against GPU time. At what width does the penalty disappear?
2. `./info` reported a subgroup size of N lanes. Does your width equal N , $\sqrt{N}$, or neither, and what does that say about the *shape* of the pixel group your GPU shades together?
3. Compute what Equation 2 predicts for the 1-pixel case and compare. If they disagree, say why they might.
4. A flat curve everywhere means `costLoops` is too small and the compiler turned both branches into arithmetic without a branch. Raise it until the curve appears and report that value.

Note that your chapter's own Lab 6 Part 3 snippet puts `x*x+1.0` against `sin(x)+1.0`, so its measured $2\times$ mixes transcendental cost into what it calls divergence cost. It also hard-codes a warp of 32. The shader above fixes both.

8 Part IV: Early-Z and draw order

Eight copies of your triangle, overlapping, with Part II's expensive fragment shader. `make earlyz` produces four numbers: two draw orders, two shader variants. Your chapter's §6.1.2 says the depth test can run *before* the fragment shader; writing `gl_FragDepth` gives that up.

Listing 5: the two `earlyz.frag` variants, given

```
1 // A: the depth test may run before this shader
2 layout(early_fragment_tests) in;
3
4 // B: the shader writes depth, so the test cannot run first
5 void main() { ...; gl_FragDepth = gl_FragCoord.z; }
```

8.1 TASK 3: pipelineState()

Six values, and the first three explain your invisible triangle in Part I.

```
1 PipelineState pipelineState(Variant v) {
2     PipelineState s{};
3     // TODO(TASK 3a): match the winding you chose in TASK 1a
4     // TODO(TASK 3b): cull back faces
5     // TODO(TASK 3c): depth compare, using Lab 05's reversed-Z convention
6     // TODO(TASK 3d): should this pipeline write depth?
7     // TODO(TASK 3e): "earlyz_a.frag" or "earlyz_b.frag", chosen from v
8     // TODO(TASK 3f): DrawOrder::FrontToBack or ::BackToFront, chosen from
9     v
10    return s;
11 }
```

The six field names are in `PipelineState`, in `src/core/Pipeline.h`. The render pass clears depth to 0.0, so choose the compare to match.

Write-up 4.

1. Your four numbers as a 2×2 table: draw order against variant.
2. Eight layers of overdraw at fullscreen: how many fragment shader invocations does the slowest cell run, and how many does the fastest avoid?
3. **A row that barely changes with draw order is a valid result**, but you must explain it: tile-based GPUs, Apple silicon among them, remove hidden surfaces before shading whatever order you submit. Which kind of GPU do you have, and what did it do to your table?

9 Part V: The index buffer

Your triangle becomes a square. Two triangles need six vertices, but only four are distinct.

9.1 TASK 4: QUAD and QUAD_INDICES

```
1 const std::vector<Vertex> QUAD = { }; // TODO(TASK 4a): four
   corners
2 const std::vector<uint16_t> QUAD_INDICES = { }; // TODO(TASK 4b): six
   indices
```

Check values. Six vertices, no index buffer: $6 \times 20 = 120$ bytes. Four vertices plus six `uint16_t`: $4 \times 20 + 6 \times 2 = 92$ bytes, a saving of 23.3%. Compute both before you look at the panel. Wind both triangles the same way as TASK 1, or the square comes out with a hole in it.

Write-up 5.

1. Your two byte totals and the saving, computed before measuring.
2. Extrapolate to the cube and sphere rows in §6.2.2. Do your numbers agree with its 73–83% claim? If not, what is different about your square?
3. Was the indexed square any *faster*? If not, explain why not, and what the saving you computed is really measuring.

10 Part VI: Name the bottleneck, prove the fix

Run `make probe`. Three scenes, each slow for a different reason, none labeled. Three sweeps tell them apart: halve the pixels, halve the instances, halve the bytes each fragment fetches. The CPU's frame time is printed beside the GPU's.

What the sweeps show	The bottleneck	The fix
Halves with pixels, flat in bytes	fragment ALU	cheaper math
Halves with pixels <i>and</i> with bytes	bandwidth	fewer fetches, smaller formats
Flat in pixels, halves with instances, CPU $\gg$ GPU	submission	one instanced draw

10.1 TASK 5: `instanceBuffer()` and `recordDraw()`

One scene issues 5,000 draw calls. You replace them with one instanced draw whose per-instance transforms live in a storage buffer the given vertex shader already reads.

```

1 static_assert(sizeof(glm::mat4) == 64, "std430 stride for an array of
   mat4");
2
3 std::vector<glm::mat4> instanceBuffer() {
4     // TODO(TASK 5a): 5000 transforms, glm::translate / glm::scale.
5 }
6
7 void recordDraw(VkCommandBuffer cmd, uint32_t indexCount) {
8     // TODO(TASK 5b): vkCmdDrawIndexed with an instance count, once.
9 }

```

Check values. In `std430` an array of `mat4` has a stride of 64 bytes, so 5,000 instances is exactly 320,000 bytes, which is what the panel prints. If every instance draws on top of the first, the vertex shader is reading one transform for all of them: check that you indexed by `gl_InstanceIndex`.

Write-up 6.

1. Classify all three scenes. For each, name the *one* sweep that decided it and quote the numbers.
2. TASK 5 before and after, for **both** the CPU frame time and the GPU pass time. Which of the two was the bottleneck before the fix, and which number says so?

3. Capture one frame with your platform's tool: `make probe` with `--capture` sets it up, RenderDoc on Windows and Linux, an Xcode `.gputrace` on macOS. Include the screenshot and name its three most expensive events.
4. No hardware needed for this one: on an H100 with 65,536 registers per SM and at most 64 warps per SM, compute the SM occupancy of a shader using 48 registers per thread. Show the steps. A 32-register shader reaches 100% and a 64-register one reaches 50%.

11 Submission

Submit your completed `student.cpp`, your filled-in `predictions.md`, and a report containing:

- **Your machine:** the `make info` screenshot. Every number below is a statement about this machine, not about GPUs in general.
- **Screenshots:** your Part I triangle, and the panel for Parts II to VI.
- **Six write-ups:** the numbered questions under each part. Answer from what is on your screen and the equations here, not from a general description of the topic. A number that contradicts the chapter, reported honestly and explained from your hardware, is a good answer; one that matches it with no evidence is not.
- **AI prompt log:** one entry per prompt, each with the **prompt**, the **model**, the **idea behind it** (what you needed and what you already knew) and the **verification** (how you checked it: working the byte offsets by hand, the check values here, the panel's read-back, reading the code).

A log entry like “wrote the triangle for me” with empty idea and verification fields does not meet this requirement. Asking AI to write code is permitted; the write-ups and those two fields are where you show your own thinking. Anything you submit should survive a follow-up question in person, and in this lab it will: the last ten minutes of the session are for exactly that.

12 References

- Akenine-Möller, T., Haines, E., Hoffman, N. (2018): *Real-Time Rendering*, 4th ed., Ch. 3 and Ch. 23. The pipeline of Section 3 and the warp behavior of Part III.
- Vulkan Specification, §Fixed-Function Vertex Processing and §Basic Polygon Rasterization. The signed-area rule behind Part I's winding check value.
- Vulkan Memory Layout appendix. The `std140` and `std430` rules of TASKs 2 and 5.
- MoltenVK Runtime User Guide. What Vulkan on macOS is, and the environment variables that produce an Xcode capture in Part VI.
- RenderDoc documentation. Part VI's capture on Windows and Linux; it does not support macOS.